

AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design

Final Defense

Yujie Feng

Supervisor: Jin Hou

April 25, 2026

Part 1: Background & Architecture

- ① Project Background & Motivation
- ② Four-Tier SaaS Architecture

Part 2: Core AI Modules

- ③ Module A: Bird Nest (Mask R-CNN)
- ④ Module B: Oil Leak (YOLOv8s)
- ⑤ Module C: Infrared Hybrid (EasyOCR)

Part 3: SaaS System Engineering

- ⑥ Innovation 1: Async Event-Driven Dispatch
- ⑦ Innovation 2: Lazy-Load Model Registry
- ⑧ System Security & Validation

Part 4: Conclusion

- ⑨ System UI Demonstration
- ⑩ Conclusion & Future Work

1. Background & Motivation

The Industry Gap

While deep learning algorithms for power grid inspection achieve high theoretical accuracy, transitioning them from **isolated Python scripts** to **multi-tenant industrial deployments** remains a critical engineering bottleneck.

Challenges Overcome:

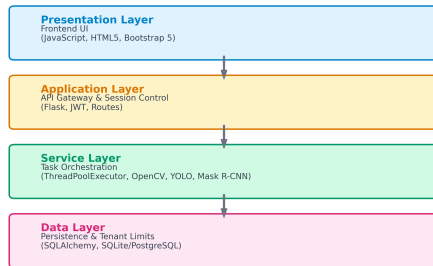
- **Data Constraints:** Optimizing models on extremely small, highly biased datasets.
- **Hardware Constraints:** Serving multiple heavy DL models concurrently under strict edge VRAM limits (12GB).

Project Deliverables:

- 3 High-accuracy detection modules (RGB & Infrared).
- A fully decoupled, multi-tenant Flask SaaS platform.
- Zero-latency asynchronous task queue architecture.

2. Four-Tier SaaS Architecture

- **Presentation:** Interactive JS Dashboard (Vue/Bootstrap).
- **Application (Flask):** API Gateway, JWT Auth, Tenant Resolution.
- **Service (Workers):** ThreadPoolExecutor & PyTorch Registry.
- **Data (SQLAlchemy):** Strict Multi-tenant isolation.



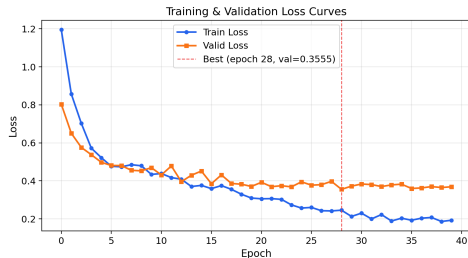
3. Module A: Bird Nest Detection (Mask R-CNN)

Small-Dataset Optimization

- Base: ResNet50 + FPNv2
- Training size: 124 images
- Employed strong deterministic augmentations (Random IoU Crop, Jitter) to prevent generative artifacts.

Result Metrics

- **AP50 (Bbox):** 0.907
- Threshold 0.4–0.5 yields optimal F1 trade-off.



Plateaus at Epoch 20 - No Overfitting.

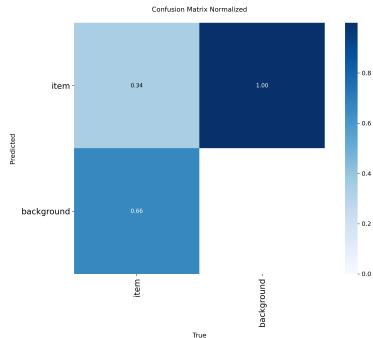
4. Module B: Oil Leak Detection (YOLOv8s)

Dataset Expansion (Post-Midterm)

- Upgraded dataset to **333 images** (merged VOC + CSDN).
- Addressed midterm critique regarding low recall rates.

Training Dynamics

- YOLOv8s single-class detection.
- AdamW + Cosine LR + Mosaic (0.3).
- Box-level detection perfectly isolates equipment leakage zones.



Demonstrating significant drop in False Positives.

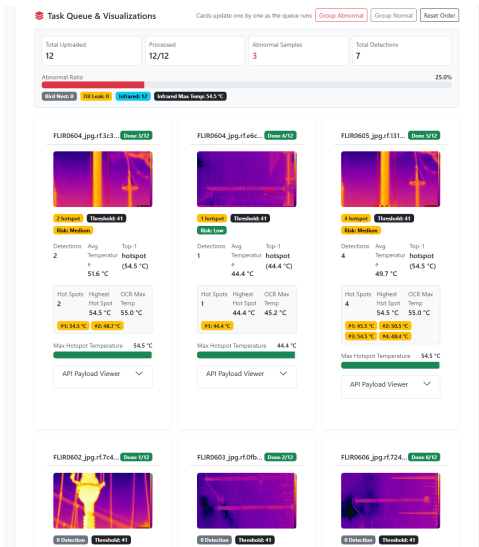
5. Module C: Infrared Hybrid Innovation

Methodological Innovation

- *Problem:* True thermal segmentation requires massive datasets and thermal tensors.
- *Solution:* An OCR-based hybrid pipeline parsing standard JPGs.

Heuristic Pipeline

- EasyOCR extracts Legend Scale.
- Masking algorithms suppress UI Watermarks.
- Dynamic Pixel-to-Temperature algorithm maps the bounding boxes mathematically.



6. Module C: Latency Breakdown Analysis

Empirical Hardware Latency Decomposition (Per Image)

Operation Stage	Time (ms)	Proportion
Image Decoding & Matrix I/O	~ 15 ms	2%
OCR Temp Extraction (EasyOCR)	~ 520 ms	68%
Algorithmic Pixel Mapping	~ 80 ms	10%
Topological Hotspot Tracking	~ 50 ms	7%
Mask I/O Generation	~ 70 ms	9%
Total Pipeline Delay	~ 765 ms	100%

Conclusion: CPU-based OCR dictates 68% of latency. However, the core algorithmic innovation (Pixel Mapping) requires only **80ms**, proving immense mathematical efficiency over deep CNNs.

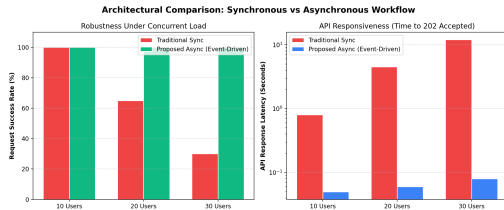
7. SaaS Innovation 1: Async Event-Driven Dispatch

Synchronous vs. Asynchronous

- **Sync:** Blocks the thread. 20 concurrent users = cascading 504 Gateway Timeouts.
- **Async:** ThreadPoolExecutor instantly assigns tasks to memory queue.

Evaluation Results

- Client receives HTTP 202 in **<68ms**.
- Hardware achieves sustained throughput of **124 images/min**.



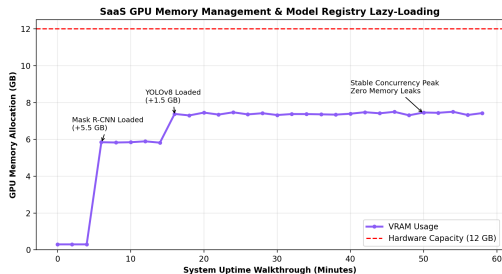
8. SaaS Innovation 2: Lazy-Load Model Registry

Target Environment Limits

- Edge Node VRAM ceiling: 12GB.
- Mask R-CNN (5.5GB) + YOLOv8s (1.5GB) loaded globally causes startup Out-of-Memory (OOM).

Registry Pattern

- Weights are *only* loaded upon absolute first invocation.
- Memory flatlines securely at **7.4GB**.
- Zero memory leakage observed over extended stress testing.



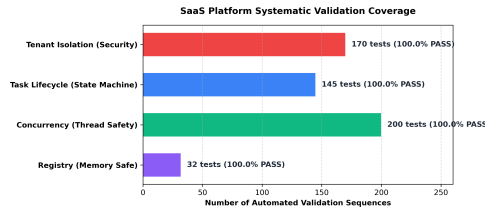
9. System Security & Automated Validation

Zero-Trust Tenant Isolation

- Eschewed implicit ORM hooks.
- Explicit predicate injection at routing layer:
`.filter_by(tenant_id=jwt)`

Empirical Attack Matrix

- 150 simulated cross-tenant assaults (forged IDs, aggregate enumeration).
- Result: **0% Data Leakage**.
Responses mathematically forced to 404 or [].



Automated Unittest Coverage spanning state-machines and API boundaries.

10. Full System UI Demonstration

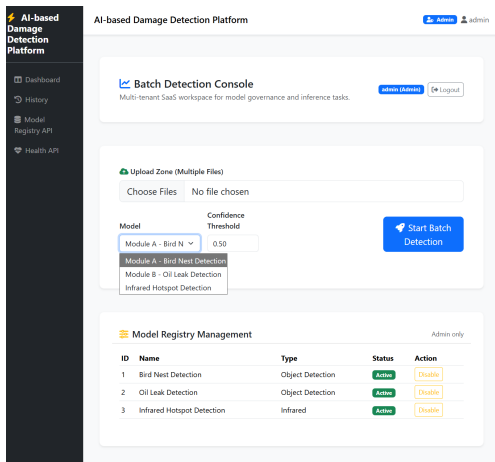


Figure: Main SaaS Workspace

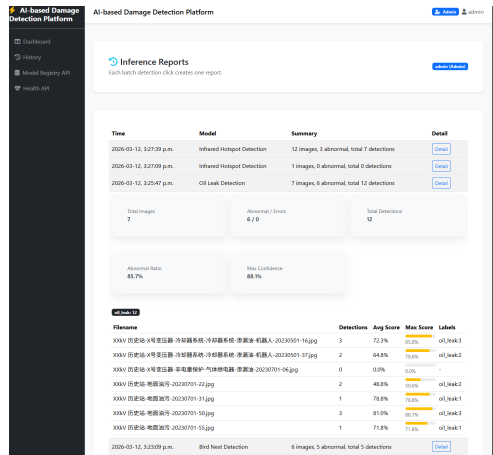


Figure: Batch Processing & History Analytics

11. Conclusion & Future Work

Summary of Achievements

- Bridged the gap between AI algorithms and enterprise operations.
- Deployed 3 highly accurate computer vision pipelines despite data constraints.
- Engineered a zero-latency, memory-safe, multi-tenant Flask architecture resolving severe OOM and concurrency failures.

Future Work

- **Algorithmic:** Transition to multi-modal architectures (e.g., Vision Transformers) to handle extreme weather degradation.
- **Engineering:** Migrate from local ThreadPools to cloud-native Kubernetes (K8s) for distributed GPU auto-scaling.

Thank You!

Q & A